

Realistic Mathematical Analysis of Modern Algorithms which Use SIMD Instructions for Better Performance

Supervisors: Prof. Cyril NICAUD
Dr. Pablo ROTONDO

Applicant: CHAU Dang Minh

Gustave Eiffel University, LIGM

Education and Research Experience

BEng in Computer Science

Univ. of Technology, Vietnam, 2024

Relevant coursework: data structures and algorithms, and compilers.

MSc in Applied Mathematics

Univ. of Limoges, 2025

Relevant coursework: stochastic optimization.

Internship: Compatibility between Open Automata (OA) and Session Types

Proved equivalence, mapped safe composition of OA to subtyping.

MSc in Mathematics & Computer Science

Gustave Eiffel Univ., in progress

Relevant coursework: combinatorics, discrete optimization and complexity theory.

Internship: Complexity Analysis of the Regular Expression Matcher [Hyperscan](#) (by Intel)

Analyze FDR string matcher (SIMD¹ extension of Shift-Or), correctness of decomposition-based matching, and the longest literal in average.

¹Single Instruction, Multiple Data

Research Experience: Analysis of Algorithms

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d) \cdot e^*$.

Hyperscan executes as follows

- 1 extracts literals from a regex ($a \cdot d$ and $b \cdot c \cdot d$ in the example),
- 2 matches all literals using FDR or another string matcher² *at once*,
- 3 verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like family³.

The longest length of the literals in average is a useful statistic.

$$Y_{n+2} \stackrel{d}{=} \underbrace{I^{(n)}}_{\text{indicator of root being concat.}} \underbrace{\left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right)}_{\text{take all of the subtrees}} + \underbrace{J^{(n)}}_{\text{indicator of root being alternation}} \underbrace{\max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right)}_{\text{take the longer of the subtrees}}, \quad Y_1 = 1, Y_2 = 0.$$

²Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.

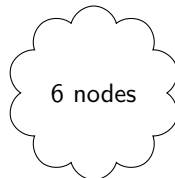
³Binary Search Tree

Research Experience: Analysis of Algorithms

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d) \cdot e^*$.

Hyperscan executes as follows

- 1 extracts literals from a regex ($a \cdot d$ and $b \cdot c \cdot d$ in the example),
- 2 matches all literals using FDR or another string matcher² *at once*,
- 3 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like family³.

The longest length of the literals in average is a useful statistic.

$$Y_{n+2} \stackrel{d}{=} \underbrace{I^{(n)}}_{\text{indicator of root being concat.}} \underbrace{\left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right)}_{\text{take all of the subtrees}} + \underbrace{J^{(n)}}_{\text{indicator of root being alternation}} \underbrace{\max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right)}_{\text{take the longer of the subtrees}}, \quad Y_1 = 1, Y_2 = 0.$$

²Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.

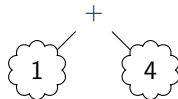
³Binary Search Tree

Research Experience: Analysis of Algorithms

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d) \cdot e^*$.

Hyperscan executes as follows

- 1 extracts literals from a regex ($a \cdot d$ and $b \cdot c \cdot d$ in the example),
- 2 matches all literals using FDR or another string matcher² *at once*,
- 3 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like family³.

The longest length of the literals in average is a useful statistic.

$$Y_{n+2} \stackrel{d}{=} \underbrace{I^{(n)}}_{\text{indicator of root being concat.}} \underbrace{\left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right)}_{\text{take all of the subtrees}} + \underbrace{J^{(n)}}_{\text{indicator of root being alternation}} \underbrace{\max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right)}_{\text{take the longer of the subtrees}}, \quad Y_1 = 1, Y_2 = 0.$$

²Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.

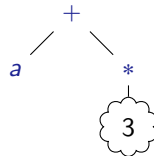
³Binary Search Tree

Research Experience: Analysis of Algorithms

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d) \cdot e^*$.

Hyperscan executes as follows

- 1 extracts literals from a regex ($a \cdot d$ and $b \cdot c \cdot d$ in the example),
- 2 matches all literals using FDR or another string matcher² *at once*,
- 3 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like family³.

The longest length of the literals in average is a useful statistic.

$$Y_{n+2} \stackrel{d}{=} \underbrace{I^{(n)}}_{\text{indicator of root being concat.}} \underbrace{\left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right)}_{\text{take all of the subtrees}} + \underbrace{J^{(n)}}_{\text{indicator of root being alternation}} \underbrace{\max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right)}_{\text{take the longer of the subtrees}}, \quad Y_1 = 1, Y_2 = 0.$$

²Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.

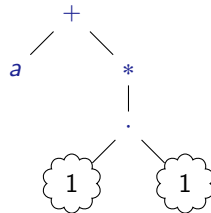
³Binary Search Tree

Research Experience: Analysis of Algorithms

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d) \cdot e^*$.

Hyperscan executes as follows

- 1 extracts literals from a regex ($a \cdot d$ and $b \cdot c \cdot d$ in the example),
- 2 matches all literals using FDR or another string matcher² *at once*,
- 3 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like family³.

The longest length of the literals in average is a useful statistic.

$$Y_{n+2} \stackrel{d}{=} \underbrace{I^{(n)}}_{\text{indicator of root being concat.}} \underbrace{\left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right)}_{\text{take all of the subtrees}} + \underbrace{J^{(n)}}_{\text{indicator of root being alternation}} \underbrace{\max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right)}_{\text{take the longer of the subtrees}}, \quad Y_1 = 1, Y_2 = 0.$$

²Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.

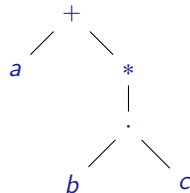
³Binary Search Tree

Research Experience: Analysis of Algorithms

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d) \cdot e^*$.

Hyperscan executes as follows

- 1 extracts literals from a regex ($a \cdot d$ and $b \cdot c \cdot d$ in the example),
- 2 matches all literals using FDR or another string matcher² *at once*,
- 3 verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like family³.

The longest length of the literals in average is a useful statistic.

$$Y_{n+2} \stackrel{d}{=} \underbrace{I^{(n)}}_{\text{indicator of root being concat.}} \underbrace{\left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right)}_{\text{take all of the subtrees}} + \underbrace{J^{(n)}}_{\text{indicator of root being alternation}} \underbrace{\max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right)}_{\text{take the longer of the subtrees}}, \quad Y_1 = 1, Y_2 = 0.$$

²Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.

³Binary Search Tree

Research Experience: Analysis of Algorithms

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d) \cdot e^*$.

Hyperscan executes as follows

- 1 extracts literals from a regex ($a \cdot d$ and $b \cdot c \cdot d$ in the example),
- 2 matches all literals using FDR or another string matcher² *at once*,
- 3 verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like family³.

The longest length of the literals in average is a useful statistic.

$$Y_{n+2} \stackrel{d}{=} \underbrace{I^{(n)}}_{\text{indicator of root being concat.}} \underbrace{\left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right)}_{\text{take all of the subtrees}} + \underbrace{J^{(n)}}_{\text{indicator of root being alternation}} \underbrace{\max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right)}_{\text{take the longer of the subtrees}}, \quad Y_1 = 1, Y_2 = 0.$$

²Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.

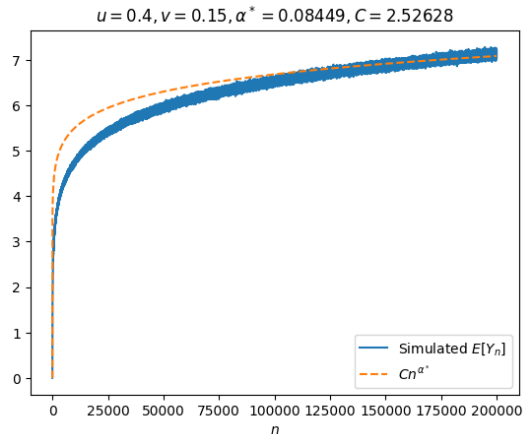
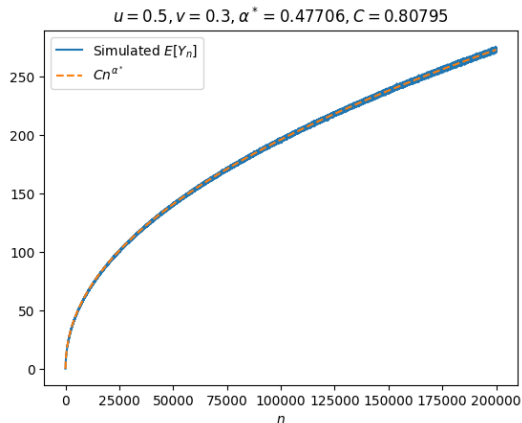
³Binary Search Tree

Research Experience: Analysis of Algorithms

For $\mathbb{E}[Y_n]$ when $2\mathbb{E}[I^{(n)}] + \mathbb{E}[J^{(n)}] := 2u + v > 1$ (agreeing with real data),

Lower bound $\Omega(n^\alpha)$ where α solves $\alpha + v2^{-\alpha} = 2(u + v) - 1$ **Upper bound** $O(n^{2(u+v)-1})$

Conjecture: in some domain of (u, v) , $\mathbb{E}[Y_n] \sim Cn^{\alpha^*}$ for some $\alpha^* \in (\alpha, 2(u + v) - 1)$.



Context and Motivation:

- Developers optimize their library algorithms in a fine-grained way e.g., they make assumptions on the input or leverage hardware features such as SIMD instructions.
- Researchers are interested in explaining if such optimizations are indeed efficient. Examples include both confirmation (TimSort⁴) and rejection (Lua's hybrid table⁵)

Goal: Develop a framework for analyzing algorithms that use SIMD.

Fruitfulness:

- Direct applications to algorithm design.
- Requirements naturally lead to deep mathematical questions, e.g., $\mathbb{E}[Y_n]$ may not be analyzable by classical techniques.

⁴Auger, N., Jugé, V., Nicaud, C., & Pivoteau, C. (2018). On the worst-case complexity of TimSort.

⁵Martínez, C., Nicaud, C., & Rotondo, P. (2022). A probabilistic model revealing shortcomings in Lua's hybrid tables.

Long-term Goal: Pursue a career as a researcher/faculty member in theoretical CS.

Short-term Goal: Secure a postdoc fellowship.

Competencies to Build During the PhD:

- Acquire technical skills in analysis of algorithms.
- Learn how to communicate research effectively, both orally and in writing.
- Write and publish findings in respected conferences and journals.
- Learn French and gain university-level teaching experience.

Thank you for your attention !